



Guerra de Cenizas

WAR OF ASHES

4X multijugador por turnos · mobile-first · la guerra como idioma de la negociación

DOCUMENTO

Refactor RTS — zonas y progresión

VERSIÓN

0.2.0

FASE

Propuesta

FECHA

2026-08-25

Universo, mecánicas y assets originales.
Documento generado desde el repositorio.

Índice

III • SISTEMAS

01 Refactor RTS — zonas y progresión

- 0. Resumen para quien no vaya a leer las 16 secciones
- 1. Qué cambia y qué no
- 2. La topología de zonas
- 3. Fronteras: Cercos, Puertas y Colosos
- 4. Extracción: Menas, Extractoras y materiales
- 5. La Ciudad en campaña: edificios con niveles
- 6. Tropas: grados
- 7. Investigación: Políticas
- 8. Cómo se roban recursos
- 9. Las dos progresiones y la regla de oro
- 10. Impacto en packages/core
- 11. Impacto en la base de datos
- 12. Impacto en la interfaz
- 13. Impacto en el simulador y los tests

01

Refactor RTS — zonas y progresión

docs/RTS_ZONES_REFACTOR.md

Versión: 2.0 · Estado: implementado en el motor y en la interfaz.

Las seis decisiones que abría están **aceptadas** (ADR-041 a ADR-046), y construirlo forzó una séptima (ADR-047) que corrige la mecánica del Coloso.

Lo que cambió al construirlo está en §19, con los números medidos en vez de estimados. Leer solo la especificación y no esa sección da una idea equivocada de cómo funciona el juego: cinco cosas de este documento resultaron estar mal, y una de ellas hacía la partida imposible de ganar.

0. Resumen para quien no vaya a leer las 16 secciones

El juego pasa de ser **un 4X de 12 turnos sobre 45-96 regiones** a ser **un 4X con capa macro de RTS, de 24 turnos, sobre 109-271 hexágonos repartidos en tres zonas concéntricas separadas por fronteras cerradas**.

Sistema	Hoy	Después
Mapa	45-96 regiones, un solo espacio continuo	109-271 hexágonos en 3 zonas con Cercos
Recursos	4 (Suministro · Industria · Intel · Ceniza)	6 (+ Mineral y Brasa , que se extraen)
Renta	Pasiva, por controlar región	Pasiva + extracción , que exige construir y mantener
Construcción	Fortificar y puentes	5 edificios con 3 niveles cada uno
Tropas	3 armas, potencia fija	3 armas × 3 grados , el grado se investiga
Investigación	No existe	Políticas : dos ramas, económica y militar
PvE	Ninguno	Colosos : guardianes deterministas que cierran las Puertas
Duración	12 turnos	24 turnos en 3 actos
Progresión permanente	Solo opciones	Solo opciones — la regla de oro no se toca (§9)

Lo que no cambia, y es la razón de que este refactor sea viable:

1. La equidad exacta por rotación C_n . Las zonas son bandas de anillos, y una rotación conserva el anillo ⇒ **la simetría sale gratis** (§2).
2. El combate determinista sin dados. Los Colosos también son deterministas.
3. `packages/core` puro, sin dependencias, sin I/O, sin reloj.
4. La niebla de guerra por `player_views`. De hecho **mejora**: un Cerco cerrado es una frontera de visión natural.
5. La premisa: ganar sigue exigiendo pagar más Ceniza de la que produce un reparto justo. Las Puertas **refuerzan** esa premisa en vez de diluirla (§3.4).

Lo que cuesta de verdad, sin adornos:

Coste	Magnitud	Dónde se trata
El motor sube de ~7 a ~13 etapas de resolución	Grande, pero aditivo	\$10
La vista de jugador crece ×4 antes de optimizar	Rompe el free tier si no se ataca	\$11
271 hexágonos no caben en 360 px	Obliga a recorrer el mapa por zonas	\$12
24 turnos rompen las cadencias asíncronas actuales	Riesgo de producto, no técnico	\$14.3
El balance vuelve a cero	~30 constantes nuevas que calibrar	\$13

Recomendación de calendario: hacerlo **antes** del despliegue público. Hoy no hay partidas reales que migrar y el coste de romper `ENGINE_VERSION` y `MAPGEN_VERSION` es cero. El día que haya una campaña en curso de una persona real, deja de serlo para siempre: una partida en curso nunca cambia de motor.

1. Qué cambia y qué no

1.1 «Mecánicas de RTS» significa la capa macro, no el tiempo real

Este es el malentendido que hay que cerrar antes de escribir una línea. Lo que se importa de un RTS es su **capa de economía**:

```
extraer → almacenar → construir → subir de nivel → investigar → producir mejor
```

Lo que **no** se importa es la ejecución en tiempo real. El juego sigue siendo por turnos, simultáneo y asíncrono; sigue resolviéndose con un `reduce(state, orders, ctx)` que corre igual en servidor, cliente y simulador. Si alguna vez alguien propone «que las unidades se muevan mientras tanto», la respuesta está aquí: eso mata el asíncrono, mata el determinismo verificable y mata la promesa de que puedes prometer un resultado exacto.

Regla. Todo lo que este documento llama «RTS» tiene que caber dentro de una orden de turno. Si una mecánica exige reaccionar entre resoluciones, no entra.

1.2 Lo que este refactor no toca

- La pureza y las cero dependencias de `packages/core`.
- El combate determinista y su previsualización exacta.
- La equidad por construcción del mapa.
- `game_states` sin políticas RLS y `player_views` filtradas por asiento.
- Que la metaprogresión de cuenta solo añada opciones (\$9).
- Que todos los assets sean SVG originales del repositorio.

1.3 Lo que sí queda invalidado

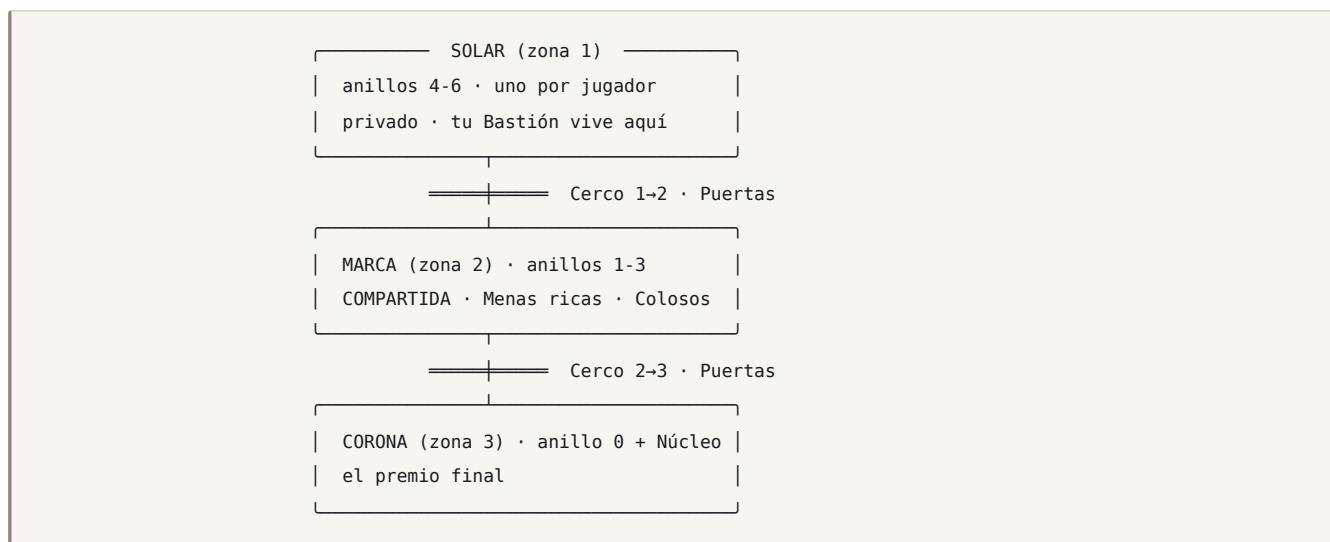
Documento	Qué deja de ser cierto
GAME_DESIGN §5, §6, §8	La economía de 4 recursos y la producción sin edificios
GAME_DESIGN §15	El orden de resolución: entran 6 etapas nuevas
MAP_GENERATION §2	SECTOR_SPEC: 4 anillos pasan a 7 y aparecen las zonas
ROADMAP v0.4–v0.8	El orden de versiones se reordena (§15)
README «12 turnos»	Pasan a 24
DISCOVERY §3	«Sin PvE» deja de valer: entran los Colosos

Cada una de esas necesita su ADR. Están propuestas en §16.

2. La topología de zonas

2.1 La idea en un dibujo

El mapa de hoy ya es concéntrico: un Núcleo en el centro y n sectores idénticos de 4 anillos cada uno. **Las zonas son bandas de anillos.** No hay que inventar geometría: hay que ponerle nombre a la que ya existe y añadirle anillos.



Visto desde arriba, con 5 jugadores: cinco Solares en la corteza, una Marca en forma de anillo que los une a todos, y una Corona en el centro.

2.2 Por qué la equidad sale gratis

Ésta es la propiedad que hace que el refactor sea barato en vez de imposible.

`zone` es una **función del anillo**:

```
function zoneOf(ring: number): Zone {
  if (ring < 0) return 3;           // el Núcleo
  return ZONE_BY_RING[ring];       // p. ej. [3,2,2,2,1,1,1]
}
```

La rotación C_n de `skeleton.ts` mapea `(sector, ring, slot) → (sector+k, ring, slot)`: **conserva el anillo**. Por tanto conserva la zona. Por tanto:

- Los n Solares son idénticos entre sí por construcción, igual que los sectores de hoy.
- La Marca es idéntica vista desde cualquier Solar.
- La Corona es equidistante de los n Bastiones.

No hay que comprobarlo con una métrica ni con un bucle de aceptación: es cierto por la misma razón que hoy lo es el reparto de yacimientos. **El refactor no gasta ni un punto de la garantía de equidad**, que es el activo más valioso del proyecto y lo único que justifica la frase «el reparto justo de un jugador».

Compárese con la alternativa evidente —zonas dibujadas a mano sobre el mapa, o zonas por distancia euclídea al centro— y se ve el ahorro: cualquiera de las dos convertiría una garantía **por construcción** en una comprobada por test, y encima en las mesas de cinco.

2.3 El tamaño nuevo

`SECTOR_SPEC` pasa de 4 anillos a 7. Propuesta  provisional:

Jugadores	Anillos por sector (interior→exterior)	Nodos/sector	Regiones	Hoy
2	[4, 5, 6, 8, 9, 10, 12]	54	109	45
3	[3, 5, 6, 8, 9, 10, 13]	54	163	55
5	[3, 4, 6, 8, 9, 11, 13]	54	271	96

Reparto por zona, con 5 jugadores:

Zona	Anillos	Nodos/sector	Total	Qué es
Corona (3)	0	3	15 + Núcleo = 16	El premio
Marca (2)	1-3	18	90	El campo de batalla real
Solar (1)	4-6	33	165	Tu casa, y tu fábrica

La regla de diseño detrás del reparto:

Tu Solar te alimenta. La Marca te hace rico. La Corona te hace ganar.

El Solar es el más grande porque es donde vive la capa de RTS —extractoras, edificios, logística— y necesita sitio para que subir de nivel sea una decisión espacial y no una lista. La Marca es la mitad de grande y contiene el triple de valor por hexágono: por eso se pelea ahí.

`sectorSize()` y `mapSize()` siguen valiendo tal cual. `assertSpecConsistency()` necesita una comprobación más: **la bolsa de terrenos se declara por zona**, no por sector, o el generador podría poner una Mena rica en un Solar.

2.4 Invariantes nuevos del esqueleto

A los 5 invariantes que ya verifica `mapgen.test.ts` se añaden 4:

- `zoneOf()` es total: toda región tiene zona, incluido el Núcleo
- Toda arista o une dos regiones de la misma zona, o es un Cerco
- Todo Solar tiene ≥ 1 Puerta hacia la Marca, y el mismo número en todos los sectores
- Ningún camino del Bastión al Núcleo evita los dos Cercos

El último es el que convierte el diseño en una regla y no en una intención: se comprueba con un BFS que ignore las aristas de Cerco y exija que el Núcleo quede **inalcanzable**.

3. Fronteras: Cercos, Puertas y Colosos

3.1 El Cerco

Un **Cerco** (`ward`) es el conjunto de aristas que separan dos zonas. Una arista de Cerco **no se puede cruzar**: no es terreno difícil, no cuesta más movimiento, no se puede rodear. Está cerrada.

En el estado se representa como un flag por arista, no como una lista aparte:

```
export interface Edge {
  a: RegionId;
  b: RegionId;
  /** Cerco: separa dos zonas. Solo se cruza por una Puerta abierta. */
  ward?: boolean;
}
```

Que sea una propiedad de la arista y no una tabla aparte es deliberado: `buildAdjacency()` ya recorre las aristas, y así el filtrado de movimiento es una condición en el sitio donde ya se decide si un salto es legal. Una tabla aparte sería una segunda fuente de verdad para la misma pregunta.

3.2 La Puerta

Una **Puerta** (`gate`) es un par de regiones adyacentes, una a cada lado de un Cerco, por el que el Cerco **puede** abrirse. Hay `gatesPerSector` Puertas por sector y por Cerco —propuesta : **1** en el Cerco 1→2 y **1** en el Cerco 2→3 —, colocadas por rotación, así que su reparto también es exacto.

Estado de una Puerta: `sealed` → `open`. **Nunca vuelve a cerrarse.**

```
export interface Gate {
  id: number;
  /** Región del lado interior (zona mayor) y del lado exterior. */
  inner: RegionId;
  outer: RegionId;
  /** Zonas que une. Siempre consecutivas. */
  from: Zone;
  to: Zone;
  open: boolean;
  /** Coloso que la guarda. `null` si ya está muerto. */
  colossus: ColossusId | null;
}
```

Que no se pueda volver a cerrar es una decisión de diseño, no una simplificación: una Puerta reversible convertiría el mapa en un sistema de compuertas donde la jugada dominante es encerrar al vecino, y encerrar a alguien es exactamente el tipo de eliminación de facto que este juego no tiene.

3.3 El Coloso

Un **Coloso** (`colossus`) es una fuerza neutral que **ocupa la región interior de una Puerta y no se mueve nunca**. Mientras vive, la Puerta está sellada. Muerto, la Puerta se abre **para todos**, no solo para quien lo mató.

Propiedad	Valor
Se mueve	No. Nunca.
Ataca	Sí: a todo lo que esté en su región al cierre del turno
Cómo pelea	Con la misma fórmula de <code>combat.ts</code> . Sin dados, sin tabla aparte
Composición	<code>line</code> / <code>fire</code> / <code>sky</code> fijos por zona  , para que la rueda de armas siga importando
Regenera	Sí, un % por turno si nadie lo tocó  — para que no se lime a picotazos gratis
Al morir	Abre la Puerta y paga el Despojo a quien dio el golpe final

Contra varios asientos a la vez reparte su daño **en proporción a la potencia de cada bando**, y los empates se rompen por número de asiento ascendente. Determinista, como todo lo demás.

La razón de diseño por la que existe —y CLAUDE.md exige que haya una— no es «que haya PvE»:

El Coloso es un **problema diplomático disfrazado de monstruo**.

Abrir la Puerta beneficia a los cinco. Pagarla la paga uno. Es un problema de bien público puesto en el sitio exacto del mapa donde el juego quiere que la gente hable, y con un precio que el motor calcula y todos pueden ver. Eso es literalmente la tesis del juego: *la diplomacia es aritmética, no social*.

3.4 La aritmética de la Puerta

Y por eso la constante más importante de todo este refactor no es el daño del Coloso:

```
Despojo(Coloso) < coste real de matarlo en bajas
```

 **Objetivo de calibración:** matar a un Coloso en solitario deja al matador **entre un 15 % y un 25 % por debajo** de donde estaba, y abre la Puerta a los otros cuatro gratis. Con dos asientos coordinados, el reparto sale a favor de los dos.

Ese único desequilibrio es el que hace que la Marca se abra por negociación y no por carrera. Si el Despojo cubriera el coste, el sistema entero degenera en «el que llegue antes», que es un 4X cualquiera.

Test que fija la intención, no el número (lección 6 de CLAUDE.md):

- ✓ abrir una Puerta en solitario deja al asiento por debajo de su estado previo
- ✓ abrirla entre dos deja a los dos por encima
- ✓ el que NO paga y entra después sigue por detrás de los que pagaron al final del acto

Ese tercer test es el que impide el problema contrario: si gorronear siempre gana, nadie abre nunca y la partida se atasca en el acto I.

4. Extracción: Menas, Extractoras y materiales

4.1 Dos recursos nuevos, y solo dos

Recurso	Código	Se obtiene de	Para qué
Mineral	<code>ore</code>	Menas de tipo <code>ore</code>	Edificios, fortificación, grados de Línea
Brasa	<code>ember</code>	Menas de tipo <code>ember</code>	Políticas, grados de Fuego y Cielo, Extractoras de nivel 3

Se para en dos a propósito. Tres materiales darían una cadena de producción más rica y **seis columnas de recursos en un HUD de 360 px**, que es donde este juego se muere. Con dos hay ya lo que hace falta:

- Ninguna zona da los dos por igual ⇒ **hay algo que comerciar**, que es lo que la diplomacia necesita para existir en el acto I, antes de que haya Sellos.
- La rama económica y la militar de las Políticas piden materiales distintos ⇒ elegir rama es elegir mapa.

Los cuatro recursos de siempre no cambian de significado. La Ceniza sigue siendo la moneda de la victoria y de la traición, y **no se extrae**: eso la mantiene escasa, que es de donde sale toda la presión del juego.

4.2 La Mena

Una **Mena** (**vein**) es un depósito **sobre un hexágono**, distinto del terreno. Un hexágono puede ser **forest** y tener una Mena de Brasa: el terreno decide el combate, la Mena decide la economía.

```
export interface Vein {
  regionId: RegionId;
  material: 'ore' | 'ember';
  /** 1-3. La riqueza sube con la zona: Solar 1, Marca 2, Corona 3. */
  grade: 1 | 2 | 3;
}
```

Una Mena no renta por controlarla. Renta si tienes una Extractora encima, la Extractora está en pie y la región está en suministro. Esa es la diferencia entre este sistema y el Yacimiento (**seam**) de hoy, que sí renta pasivamente y **se queda como está**: son dos cosas distintas con dos propósitos distintos, y conviene no mezclarlas.

	Yacimiento (seam)	Mena (vein)
Da	Ceniza	Mineral o Brasa
Exige	Controlar la región	Controlar y construir y mantener
Se destruye	No	La Extractora sí
Se roba	No	Sí, con su almacén (\$8)
Para qué sirve	Ganar	Creecer

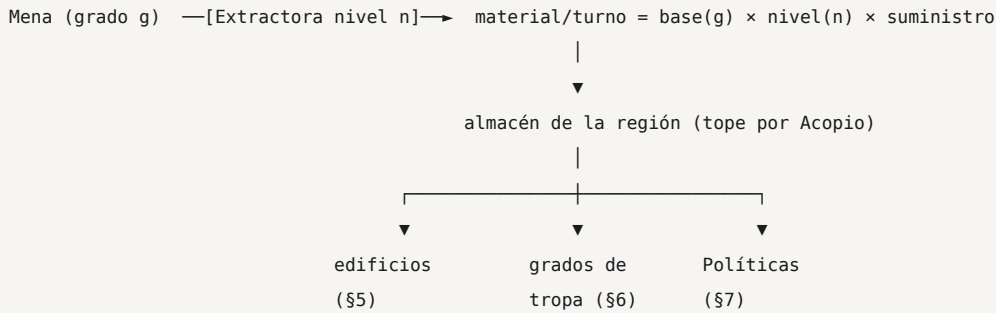
4.3 Reparto de Menas por zona

 Provisional, por sector:

Zona	Menas	Grado	Materiales
Solar	6	1	3 Mineral + 3 Brasa — simétrico a propósito : tu casa nunca te obliga a comerciar
Marca	6	2	asimétrico por sector : 4/2 y 2/4 alternando ⇒ hay algo que negociar
Corona	2	3	1 + 1

La asimetría de la Marca **no rompe la equidad C_n**: el patrón se replica por rotación, así que todo asiento tiene exactamente el mismo reparto **relativo a su propio Solar**. Lo que cambia es a qué vecino le sobra lo que a ti te falta, y eso es geografía diplomática, no ventaja.

4.4 La cadena



Tres decisiones de diseño que van juntas:

1. **El material se almacena en la región, no en el asiento.** Por eso se puede robar (§8) y por eso la logística importa. Un imperio con todo el Mineral en una Extractora de la Marca es un imperio con un problema.
2. **La extracción exige suministro.** Una Extractora sin suministro no produce. Eso reconecta la capa de RTS con el freno anti-*snowball* que ya existe, en vez de crear una economía paralela que lo esquive.
3. **El material se recoge al pasar por casa,** no teletransportado: el traslado del almacén al asiento ocurre en la etapa de logística, y consume una fracción por salto 🏰.

5. La Ciudad en campaña: edificios con niveles

5.1 Los cinco edificios

Cinco, con niveles 1-3. Ni uno más: cada edificio nuevo es una columna más en una ficha de región que ya se lee en 360 px de ancho.

Edificio	Código	Dónde	Qué hace al subir
Extractora	<code>extractor</code>	Sobre una Mena	Más material por turno
Fundición	<code>foundry</code>	Bastión y urbanas	Convierte Mineral → Industria; techo del grado de tropa
Arsenal	<code>arsenal</code>	Bastión	Abarata producción y sube el tope de fuerzas
Acopio	<code>depot</code>	Cualquiera propia	Sube topes de almacén y alcance de suministro
Atalaya	<code>watch</code>	Alta y urbana	Visión y generación de Intel

```
export type BuildingKind = 'extractor' | 'foundry' | 'arsenal' | 'depot' | 'watch';

export interface Building {
  regionId: RegionId;
  kind: BuildingKind;
  /** 1-3. No hay nivel 0: un edificio de nivel 0 es un edificio que no existe. */
  level: 1 | 2 | 3;
  /** Turnos que faltan para terminar la obra en curso. 0 = operativo. */
  building: number;
}
```

5.2 Las reglas que evitan que esto sea una hoja de cálculo

Un sistema de edificios con niveles es la vía más rápida conocida para convertir un juego en una lista de botones que se pulsan en orden. Cuatro reglas lo impiden:

1. **Una obra por región y turno.** No hay colas de producción — ya se descartaron en el brief, y por la misma razón: una cola es una decisión que tomas una vez y el juego ejecuta sin ti.
2. **Subir de nivel tarda turnos (1 / 2 / 3 ) y durante la obra el edificio no produce.** Mejorar es renunciar a renta ahora por renta después: eso es una decisión.
3. **Los edificios se capturan, no se destruyen.** Quien toma la región se queda el edificio en su nivel, menos uno . Así atacar una Extractora de nivel 3 es mejor que construirse la propia, y el mapa se pelea.
4. **El techo lo pone la Fundición.** No puedes tener un Arsenal de 3 con una Fundición de
 1. Una sola dependencia, para que haya un orden que descubrir sin que haya un árbol que memorizar.

5.3 Qué decisión interesante permite

¿Subo la Extractora del Solar, que es segura y da poco, o la de la Marca, que da el triple y puede que mañana sea de otro?

Esa pregunta no existe hoy y es exactamente el tipo de decisión que CLAUDE.md exige antes de dejar entrar un sistema. La respuesta correcta depende de con quién hayas hablado esta mañana, que es donde el juego quiere que estén todas las respuestas.

6. Tropas: grados

6.1 El sistema

Cada arma —Línea, Fuego, Cielo— tiene un **Grado** () de 1 a 3 por asiento y por campaña. El grado multiplica la potencia de las unidades **producidas a partir de ese momento**; las que ya están en el mapa **no se actualizan solas**.

```
/** Índice = arma. Grado actual del asiento, 1-3. */
export interface SeatState {
  // ...
  tiers: { line: 1 | 2 | 3; fire: 1 | 2 | 3; sky: 1 | 2 | 3 };
}
```

Grado	Multiplicador 	Coste 	Requiere
1	1,00	—	—
2	1,25	40 Mineral + 20 Brasa	Fundición 2
3	1,55	90 Mineral + 60 Brasa	Fundición 3

Que las unidades viejas no se actualicen es la decisión que sostiene el sistema entero. Si se actualizaran, subir de grado sería una mejora global sin contrapartida y la única pregunta sería «¿cuándo puedo pagarla?». Sin actualización retroactiva aparece la pregunta buena:

Tengo 60 de Línea de grado 1 en el frente. ¿Subo a grado 2 y empiezo a reemplazar, o me gasto lo mismo en más grado 1 y ataco este turno?

6.2 Por qué el grado no rompe la rueda de armas

El multiplicador se aplica **antes** de la rueda `counterK`, no después. Una Línea de grado 3 sigue perdiendo contra Fuego de grado 3 exactamente en la misma proporción que en grado

1. El grado sube el suelo, no rompe el piedra-papel-tijera.

Test bloqueante: para todo par de grados (g_1 , g_2), la matriz de resultados de la rueda conserva el signo. Si se rompe, el juego pasa a tener una composición dominante y el criterio de aceptación de v0.2 —*ninguna composición monoarma vence a todas las demás*— deja de cumplirse.

7. Investigación: Políticas

7.1 Dos ramas, seis nodos, tres niveles

Una **Política** (`policy`) es una mejora que, una vez investigada, **se aplica durante el resto de la campaña** y no se puede deshacer.

RAMA ECONÓMICA (Brasa)		RAMA MILITAR (Mineral)	
└ Vetas Profundas	×1,15 extracción	└ Cadencia	×1,10 potencia de Fuego
└ Caravanas	-20 % pérdida	└ Escalada	+1 nivel efectivo de fort. al asaltar
	por salto		
└ Refundición	-25 % coste de edificios	└ Doctrina de Marcha	-30 % coste de suministro fuera del Solar

Seis nodos, tres niveles cada uno . Investigar consume material y **un turno de la Fundición**, así que investigar y construir compiten por el mismo edificio: no se puede hacer todo.

7.2 «Permanente» significa el resto de la campaña

Es el punto donde este documento se juega su coherencia con el resto del proyecto, así que conviene decirlo sin ambigüedad:

Las Políticas son permanentes **dentro de la campaña**. Al terminar, se pierden como se pierde todo lo demás de la partida.

Lo que la cuenta se lleva a casa es **qué Políticas existen en tu árbol**, no en qué nivel las dejaste. La razón está en §9, y no es purismo: es el único test que separa este juego de un *pay-to-win* con otro nombre.

7.3 Por qué políticas y no un árbol tecnológico

El brief ya descartó el árbol tecnológico grande, y por un motivo que sigue vigente: un árbol de 40 nodos es una tarea de memorización, no una decisión. Seis nodos en dos ramas que compiten por el mismo edificio y por los mismos materiales caben en una pantalla, se entienden a la primera y **obligan a elegir** — que es lo único que se le pedía al sistema.

8. Cómo se roban recursos

La petición era explícita: los recursos se obtienen «durante la guerra, a través de menas, bosses de zona, robando al enemigo». Las cuatro vías, con su nombre canónico:

Vía	Nombre	Cómo
Extraer	Extracción	Extractora sobre Mena, en suministro
Matar un Coloso	Despojo (spoils)	Al golpe final. Menos de lo que cuesta (§3.4)
Tomar una región	Captura	Te quedas el almacén de la región y el edificio con un nivel menos
Atacar sin tomar	Botín (plunder)	Ganas el combate, te llevas un % del almacén y te retiras

8.1 El Botín es una orden, no un efecto secundario

`plunder` es una **postura** nueva, hermana de Asalto / Firme / Pantalla:

```
export type Posture = 'assault' | 'hold' | 'screen' | 'plunder';
```

En postura Botín una fuerza ataca con una penalización ⚖️ (–25 % de potencia), y si gana: se lleva un porcentaje del almacén de la región ⚖️ (40 %), **no captura la región y vuelve a su casilla de origen**. Si pierde, pierde como cualquiera.

Tres razones por las que esto es mejor que un «robo» automático al capturar:

1. Es una **decisión declarada de antemano**, y por tanto se puede prometer y se puede mentir sobre ella. Que es de lo que va el juego.
2. Da una jugada a quien va perdiendo. Un asiento sin territorio para expandirse todavía puede hacer daño y sigue teniendo algo que ofrecer en una negociación. Sin eso, el turno 15 de un jugador que va último es no hacer nada durante nueve turnos.
3. Convierte el almacén de la región en una **posición defendible**, lo que da a las Extractoras de la Marca una tensión que no tendrían si el material fuera directamente al asiento.

9. Las dos progresiones y la regla de oro

9.1 El conflicto, dicho de frente

La petición incluye «las tropas se mejoran con metaprogresión» e «investigación general para mejorar políticas [...] que se aplican de forma permanente». Leído literalmente, eso significa que **una cuenta veterana empieza la campaña con números mejores que una cuenta nueva**.

Eso choca de frente con la regla nº 4 de `CLAUDE.md` y con **METAPROGRESSION §2**, que no son una preferencia estética sino un test bloqueante de CI:

```
✓ simulación: cuenta al 100 % vs. cuenta vacía con la misma doctrina y anomalías
⇒ winrate dentro de 48-52 % en 2 000 partidas
```

Si la metaprogresión toca números, ese test **no puede pasar por definición** y hay que borrarlo. Y borrado, el juego deja de poder prometer que una mesa de cinco es una mesa justa, que es la premisa que sostiene la diplomacia aritmética: nadie negocia un reparto justo si el reparto de salida ya no lo era.

9.2 La propuesta: sube el árbol, no el nivel

Recomendada. La regla de oro se mantiene intacta y se extiende a los sistemas nuevos:

Lo que la campaña da (números)	Lo que la cuenta guarda (opciones)
Niveles de edificio 1→3	Qué edificios puedes construir
Grados de tropa 1→3	Qué especializaciones de grado existen para ti
Niveles de Política	Qué Políticas aparecen en tu árbol
Materiales acumulados	Nada: se pierden

Toda campaña empieza con **todos los edificios a nivel 1, todas las tropas a grado 1 y cero Políticas investigadas**, para la cuenta nueva y para la veterana. Lo que el veterano tiene es un **abanico más ancho**: elige 6 Políticas de un catálogo de 12 en vez de tener 6 fijas, y puede adaptar el equipo al número de jugadores, al perfil del mapa y a quién se sienta enfrente.

Es exactamente el modelo que el proyecto ya eligió para doctrinas y anomalías, aplicado a los sistemas nuevos. Y sigue sintiéndose a progresión, porque **lo es**: es la progresión de un juego de cartas con mazo construido, no la de un RPG con niveles.

9.3 La alternativa, por si la decisión es la contraria

Si el dueño del proyecto quiere de todas formas progresión numérica persistente —es una decisión de producto legítima; media industria vive de ella— lo que **no** se puede hacer es mezclarla con la mesa competitiva. La forma de tenerla sin destruir la premisa es partir el juego en dos ligas:

	Guerra (por defecto)	Guerra de Legado (opcional)
Progresión numérica persistente	No	Sí: niveles de Ciudad que entran a campaña
Emparejamiento	Libre	Por franja de Renombre , obligatorio
Regla de oro	Intacta	No aplica; el test bloqueante se sustituye
Test de winrate	48-52 % cuenta llena vs. vacía	48-52 % dentro de la franja

Coste honesto de la alternativa: dos conjuntos de constantes que calibrar, un emparejamiento por franjas que hoy no existe (y que necesita **volumen de jugadores** que un juego en beta cerrada no tiene), y una segunda liga que dividirá una comunidad pequeña. Por eso la recomendación es §9.2 — pero la decisión no es técnica y no es mía.

Ésta es la única decisión de este documento que bloquea trabajo. Todo lo demás se puede empezar sin resolverla; §5, §6 y §7 se implementan igual en los dos escenarios, porque en los dos los números viven en la campaña. Lo único que cambia es de dónde salen los valores iniciales.

10. Impacto en `packages/core`

10.1 Tipos

```
// types/index.ts – añadidos

export type Zone = 1 | 2 | 3;
export type MaterialId = 'ore' | 'ember';
export type ColossusId = string;

export interface Region {
  // ...lo de hoy...
  zone: Zone; // derivado del anillo, guardado por comodidad de la vista
}

export interface Edge { a: RegionId; b: RegionId; ward?: boolean }

export interface GameMap {
  // ...lo de hoy...
  gates: Gate[];
  veins: Vein[];
}

export interface Resources {
  supply: number; industry: number; intel: number; ash: number;
  ore: number; ember: number; // ← nuevos
}

export interface SeatState {
  // ...lo de hoy...
  tiers: Record<'line'|'fire'|'sky', 1|2|3>;
  policies: Record<PolicyId, 0|1|2|3>;
}

export interface GameState {
  // ...lo de hoy...
  buildings: Building[]; // ordenado por (regionId, kind)
  stock: Record<MaterialId, number>[]; // índice = regionId. El almacén de cada región
  colossi: Colossus[];
  gatesOpen: boolean[]; // índice = gate.id
}
```

Colossus va en su propio array y NO reutiliza **Force**. Es la decisión de tipos más importante del refactor: hacer `Force.seat` anulable para meter neutrales obligaría a revisar `control.ts`, `economy.ts`, `views.ts` y todos los desempates por asiento, y convertiría un cambio aditivo en uno que toca cada `switch` del paquete. Un array aparte con su propia etapa cuesta 40 líneas y no toca nada.

10.2 El orden de resolución

De 7 etapas efectivas a 13. Los huecos numerados del GDD §15 se llenan; el pipeline sigue siendo **una función pura por etapa** y añadir un sistema sigue siendo insertar una etapa.

1 · Validación	← + órdenes de obra, grado y política
2-4 · Diplomacia / anomalías / Sombra	(siguen vacías: v0.6+)
5 · Movimiento simultáneo	← + filtro de Cerco
6 · Combate entre asientos	
6b· COMBATE CONTRA COLOSOS	← NUEVA
6c· BOTÍN Y RETIRADA	← NUEVA
7 · Control territorial	← + captura de edificios y almacenes
7b· APERTURA DE PUERTAS	← NUEVA
8 · Economía: renta, mantenimiento, suministro	
8b· EXTRACCIÓN	← NUEVA
8c· LOGÍSTICA: almacén → asiento	← NUEVA
9 · Producción	← + grados de tropa
9b· OBRAS: avance y finalización	← NUEVA
9c· INVESTIGACIÓN	← NUEVA
10-11 · Núcleo y anomalías de información (v0.7+)	
14 · Cierre de turno	

El orden de 6b, 6c y 7b importa y no es negociable: un Coloso muere **después** del combate entre asientos —para que dos asientos puedan pelearse por el golpe final— y la Puerta se abre **después** del control, para que abrirla no cambie quién controla qué en el mismo turno en que se abre.

10.3 Módulos

Archivo	Estado
<code>rules/zones.ts</code>	nuevo — <code>zoneOf</code> , filtro de Cerco, apertura de Puertas
<code>rules/colossus.ts</code>	nuevo — combate contra neutrales, Despojo, regeneración
<code>rules/extraction.ts</code>	nuevo — Menas, Extractoras, almacenes, logística
<code>rules/buildings.ts</code>	nuevo — obras, niveles, captura, techo por Fundición
<code>rules/research.ts</code>	nuevo — grados y Políticas
<code>rules/movement.ts</code>	filtro de Cerco en la validación de saltos
<code>rules/battle.ts</code>	postura Botín; los Colosos no entran aquí
<code>rules/combat.ts</code>	multiplicador de grado antes de la rueda. Nada más
<code>rules/economy.ts</code>	6 recursos; el suministro consulta el Acopio
<code>rules/control.ts</code>	la captura arrastra edificios y almacén
<code>rules/views.ts</code>	proyección de zona, Menas visibles, edificios ajenos
<code>mapgen/spec.ts</code>	7 anillos, zonas, bolsas por zona, Puertas
<code>mapgen/generate.ts</code>	siembra de Menas y colocación de Colosos
<code>balance/constants.ts</code>	~30 constantes nuevas 

`combat.ts` se toca lo mínimo **a propósito**: es el módulo que comparten la resolución y la previsualización, y cualquier estado de turno que entre ahí rompe que el pronóstico coincida exactamente con el resultado. El multiplicador de grado puede entrar porque es un dato del asiento, no del turno.

10.4 Versionado

`ENGINE_VERSION` y `MAPGEN_VERSION` suben los dos, y es **incompatible hacia atrás**: una partida creada con el motor de hoy no se puede resolver con el nuevo, ni al revés. No hay migración posible ni conviene intentarla — el estado cambia de forma, no de contenido.

Consecuencia operativa, y es la que fija el calendario: **este refactor tiene que entrar antes de que haya una sola campaña real en curso.**

11. Impacto en la base de datos

11.1 El problema que aparece al multiplicar por cuatro

`player_views` guarda una fila `(game_id, turn, seat)` con la vista **entera** en `jsonb`, y `PlayerView` incluye `map: GameMap`. Es decir: **hoy el mapa completo se serializa una vez por asiento y por turno.**

Con el mapa de hoy eso es tolerable. Con el nuevo, no:

	Hoy (5p, 12 turnos)	Refactor sin optimizar	Con §11.2
Regiones	96	271	271
Vista por asiento y turno	~15 KB	~62 KB	~24 KB
Filas por campaña	60	120	120
Por campaña	~0,9 MB	~7,4 MB	~2,9 MB
Campañas en 500 MB	~550	~67	~170

🔗 Estimaciones a partir de la forma actual de `PlayerView`, no medidas. **Hay que medirlas antes de aceptar el ADR**, porque de este número depende que el objetivo de 0 €/mes durante la beta siga siendo posible.

11.2 El mapa deja de viajar

El mapa es **inmutable durante toda la campaña**. No hay razón para reenviarlo 120 veces.

```
games.map_id      → game_maps(id, mapgen_version, seed, players, map jsonb)
player_views.view → la vista SIN `map`, más `mapId`
```

El cliente pide el mapa una vez y lo cachea; la vista trae solo lo que cambia. Rebaja la vista un 60 % y —esto es lo que la hace obligatoria y no una optimización— **es independiente del refactor**: merece hacerse igualmente, y hacerla ahora es más barato que hacerla con partidas en producción.

Cuidado con la RLS: `game_maps` es legible por **cualquier asiento de esa partida**, y por nadie más. El mapa es público entre los cinco (la topología no es secreto; lo son las fuerzas), pero eso hay que escribirlo como política, no darlo por hecho.

11.3 Lo que las zonas regalan a la niebla de guerra

Buena noticia, y no menor: **un Cerco cerrado es una frontera de visión**. Mientras la Puerta esté sellada, un asiento no ve nada de la Marca más allá de la propia Puerta. La vista del acto I es, por tanto, **más pequeña que la de hoy** pese a que el mapa sea el triple: solo tu Solar y el borde.

La vista crece al abrirse cada Cerco, que es justo cuando la partida se pone interesante y cuando ya quedan menos turnos. El pico de datos se desplaza al final y se aplana.

11.4 Migraciones nuevas

0012_map_store.sql	game_maps + games.map_id + RLS por asiento
0013_zones.sql	nada de estado – es todo jsonb dentro de game_states
0014_view_shrink.sql	player_views sin `map`

El estado de juego sigue viviendo en un solo `jsonb`: partir `buildings`, `stock` o `colossi` en tablas propias sería exponer el estado autoritativo a PostgreSQL y **tirar la niebla de guerra por el desagüe**. Sigue en pie la regla nº 3.

12. Impacto en la interfaz

12.1 271 hexágonos no caben en 360 px, y no hay truco

Ya está medido: con 96 regiones, a escala 1 cada región mide **21 px**, muy por debajo de los 44 px de objetivo táctil. Con 271 caerían a ~12 px. No es un problema de zoom: es que **el mapa entero deja de ser una vista útil**.

La solución no es dibujar más pequeño, es **dejar de dibujarlo entero**:

NIVEL 1 · Vista de zonas	3 anillos esquemáticos con cifras agregadas. Sin hexágonos. Sirve para decidir A DÓNDE mirar.
NIVEL 2 · Vista de zona	Una zona, o un Solar. 33-90 hexágonos a ≥ 44 px. Es donde se juega. Sólo esto está en el DOM.
NIVEL 3 · Ficha de región	Lo de hoy: acciones con nombre, pronóstico, obras.

Solo el nivel 2 tiene regiones en el DOM, y como mucho ~90 elementos enfocables — menos que los 96 de hoy. Se conserva por tanto todo lo de **ADR-034** y **ADR-040**: cada región sigue siendo un `<path>` enfocable y anunciante, y el presupuesto de render no empeora.

Lo que se pierde es la panorámica, y hay que decirlo: hoy se puede abarcar el mapa entero de un vistazo y después de esto no. A cambio se gana que cada hexágono sea tocable de verdad, que es la diferencia entre un mapa y una ilustración.

12.2 Lo que las zonas hacen fácil

La navegación por zonas resuelve gratis un problema que hoy se resuelve a mano: el encuadre. «Ir a mi Solar», «ir a la Marca», «ir a la Puerta norte» son tres destinos con nombre, no tres arrastres. El botón que devuelve la cámara al Bastión —que se añadió precisamente porque perder la ciudad de vista arruinaba la partida— pasa a ser un caso particular de algo general.

12.3 El HUD con seis recursos

Cuatro cifras caben en 360 px. Seis, no, si todas tienen el mismo peso. Reparto propuesto:

Barra permanente	Suministro · Industria · Ceniza	(lo que se gasta cada turno)
Bajo demanda	Mineral · Brasa · Intel	(lo que se acumula)

Mineral y Brasa se enseñan **en el sitio donde se deciden**: en la ficha de región con Extractora, en la pantalla de obras y en la de Políticas. Un número que solo importa cuando vas a gastarlo no necesita estar en pantalla los otros 23 turnos.

12.4 Pantallas nuevas

Pantalla	Dónde	Regla
Vista de zonas	Nivel 1 del mapa	No es un menú: es el mapa alejado
Obras	Ficha de región	Botones con nombre, como el resto (ADR-038)
Políticas	Panel propio, hermano del mapa	No flota sobre el mapa. Ocupa sitio
Grados	Dentro de Producción	Es una decisión de producción, no un menú aparte

La regla que las gobierna a las cuatro está aprendida a base de repetirla en este repositorio: **ningún panel flota sobre el mapa**. `pointer-events: none` resuelve los taps, no la visibilidad.

13. Impacto en el simulador y los tests

13.1 Tests nuevos, por bloque

zonas

- ✓ `zoneOf()` es total y coincide con la banda de anillos
- ✓ toda arista intra-zona no es Cerco; toda inter-zona lo es
- ✓ el Núcleo es INALCANZABLE ignorando las aristas de Cerco
- ✓ los `n` Solares son idénticos por rotación (inventario y Menas)
- ✓ toda Puerta tiene su imagen rotada en los `n` sectores

colosos

- ✓ un Coloso nunca se mueve
- ✓ reparte daño en proporción, y empata por asiento ascendente
- ✓ al morir abre la Puerta PARA TODOS, no solo para el matador
- ✓ el Despojo NO cubre el coste de matarlo en solitario ← intención, no número
- ✓ dos asientos coordinados salen ganando ← intención, no número
- ✓ quien no paga y entra después sigue por detrás al cierre del acto

extracción

- ✓ una Mena sin Extractora no produce nada
- ✓ una Extractora sin suministro no produce nada
- ✓ el almacén vive en la región y se captura con ella
- ✓ el Botín deja la región en manos del defensor

edificios y grados

- ✓ una obra en curso no produce
- ✓ el techo de la Fundición se respeta en toda ruta de construcción
- ✓ subir de grado NO actualiza las unidades ya desplegadas
- ✓ la rueda de armas conserva el signo para todo par de grados ← BLOQUEANTE

progresión

- ✓ toda campaña empieza a nivel 1, grado 1 y cero Políticas
- ✓ no-power-creep sigue en verde con los sistemas nuevos ← BLOQUEANTE

presupuesto

- ✓ una `PlayerView` de 271 regiones serializa por debajo del tope ← BLOQUEANTE

Los tres bloqueantes del proyecto pasan de tres a cinco: se les suman la rueda con grados y el tope de tamaño de la vista.

13.2 El simulador deja de ser opcional

Hoy `packages/sim` está sin implementar y llega en v0.8. Con este refactor eso deja de ser sostenible: se pasa de ~20 constantes de balance a ~50, y varias de ellas —el Despojo, el coste de Puerta, los multiplicadores de grado— **no se pueden calibrar a mano** porque su efecto es de segundo orden y aparece en el turno 14.

El simulador se adelanta y pasa a ser prerequisite de la primera versión del refactor, no de la última.

Con una salvedad que ya está escrita en su `CLAUDE.md` y que aquí importa el doble: el simulador mide si la **aritmética** de la Puerta funciona; no mide si un Coloso es divertido. Eso solo lo dice el playtesting.

13.3 Métricas nuevas del informe

Métrica	Objetivo 
Turno de apertura del primer Cerco 1→2	7-10
Turno de apertura del primer Cerco 2→3	16-19
% de campañas donde la primera Puerta la pagan ≥ 2 asientos	> 55 %
% de campañas que llegan a la Corona	> 85 %
Reparto de material entre asientos, Gini al T18	< 0,30
corr(Menas@T12, victoria)	< 0,45

La tercera es la que dice si el diseño funciona: **si la mayoría de las Puertas las abre un solo asiento, el Coloso es un peaje y no un problema diplomático**, y hay que subir su coste hasta que lo sea.

14. Presupuestos

14.1 Datos

Presupuesto	Valor
<code>PlayerView</code> serializada, sin <code>map</code>	≤ 30 KB
<code>GameState</code> serializado	≤ 180 KB
Campaña completa en BD	≤ 3 MB
Egreso por campaña jugada	≤ 4 MB

14.2 Render

Presupuesto	Valor
Regiones en el DOM a la vez	≤ 96 (igual que hoy)
JS de la ruta de partida (gzip)	≤ 180 KB (sin cambio)
INP al tocar una región	≤ 100 ms (sin cambio)
Cambio de zona	≤ 250 ms

Los tres primeros no se relajan. Que el mapa sea tres veces mayor **no es una excusa para gastar más**: es la razón por la que hay que recorrerlo por zonas.

14.3 Duración

Aquí está el riesgo de producto del refactor.

Cadencia	Hoy (12 turnos)	Con 24 turnos	Propuesta
Blitz	~50 min	~100 min	Turno de 3 min ⇒ ~72 min
Diaria	~6 días	~12 días	2 turnos/día ⇒ ~12 días
Relajada	~12 días	~24 días	~24 días, y sobra

Una campaña asíncrona de 24 días **no la termina nadie**. Tres salidas, y hay que elegir una antes de fijar

`BALANCE.campaign.turns`:

1. **Menos turnos, actos más densos** — 18 en vez de 24. Recomendada: la duración está para servir a los actos, no al revés.
2. **Dos turnos al día en la cadencia diaria** — funciona, pero cambia el contrato con el jugador («juego una vez al día») que es de donde sale el asíncrono.
3. **Aceptar 24 días en Relajada y quitar la Diaria** — la más honesta y la que más jugadores pierde.

⚖ Este documento asume 24 turnos para dimensionar lo demás, pero **la cifra es lo primero que el simulador debe atacar**: si con 18 turnos se abren los dos Cercos y la Corona se disputa, 18 es mejor número que 24 por razones que no tienen nada que ver con el diseño.

15. Plan de versiones

El refactor no cabe en una versión. Se parte en cinco, cada una **jugable de principio a fin**, porque la regla del proyecto es que ninguna versión deja deuda a la siguiente.

Y se pone **antes** de la diplomacia, no después: la diplomacia de v0.4 se construye sobre la economía y el mapa, y construirla dos veces cuesta más que retrasarla una.

Versión	Nombre	Alcance	Criterio que la cierra
v0.4	Zonas	7 anillos, Cercos, Puertas, <code>zoneOf</code> , mapa por zonas en la UI	Una campaña completa en un mapa de 271 regiones, con los dos Cercos abiertos a mano en modo depuración
v0.5	Extracción	Menas, Extractoras, materiales, almacenes, logística, Botín	Un asiento puede financiar su guerra solo con extracción
v0.6	Colosos	PvE determinista, Despojo, apertura de Puertas	La aritmética de la Puerta se cumple en simulación
v0.7	Ciudad de campaña	5 edificios × 3 niveles, grados, Políticas	Ninguna ruta de construcción domina en 2 000 partidas
v0.8	Balance	~50 constantes calibradas con barridos guardados	Los tres tests bloqueantes, más los dos nuevos

Lo de después —Diplomacia, Núcleo, Metaprogresión, Anomalías— se corre cuatro números y **no cambia de contenido**. La única alteración de fondo es que el simulador se adelanta de v0.8 a prerequisite de v0.4.

16. Decisiones que este documento abre

Seis, todas registradas en [DECISIONS.md](#) y todas en estado **propuesta**:

ADR	Decide	Sustituye o matiza
ADR-041	Las zonas son bandas de anillos, y por eso la equidad C_n no se toca	Amplía ADR-037
ADR-042	Solo una zona vive en el DOM: el mapa grande se recorre, no se abarca	Matiza ADR-040
ADR-043	El Coloso es un problema diplomático disfrazado de monstruo	Contradice «sin PvE» de DISCOVERY §3
ADR-044	El mapa deja de viajar en cada vista	Amplía ADR-028
ADR-045	La metaprogresión sigue sin tocar números: sube el árbol, no el nivel	Confirma METAPROGRESSION §2
ADR-046	La campaña se juega en tres actos, y la duración la fija el simulador	Cambia <code>BALANCE.campaign.turns</code>

Solo ADR-045 bloquea. Las otras cinco se pueden aceptar y empezar en cualquier orden.

17. Riesgos

#	Riesgo	Probabilidad	Mitigación
1	La vista no baja de 30 KB y el free tier revienta	Media	Medirlo en v0.4 con un mapa real antes de escribir la UI. Si no baja, la vista pasa a delta por turno
2	271 hexágonos se leen peor que 96 aunque cada uno sea tocable	Media	La vista de zonas es lo primero que se prueba en 360×640, antes que el motor
3	24 turnos no los termina nadie en asíncrono	Alta	§14.3 : decidir la cifra con el simulador, no con el diseño
4	El Coloso se convierte en un peaje y nadie negocia	Media	La métrica de «Puertas pagadas por ≥ 2 asientos» está en el informe desde el primer día
5	La capa de RTS tapa la diplomacia : 24 turnos gestionando obras y ninguno hablando	Alta	Techo duro de 5 edificios y 6 Políticas. Toda ampliación entra por ADR con algo que se corta a cambio
6	El balance no converge con 50 constantes	Alta	El simulador se adelanta a prerrequisito. Sin él, no se empieza
7	Se despliega antes de refactorizar y hay partidas que migrar	Baja	No hay migración: se drena. Por eso el refactor va antes del despliegue

El riesgo 5 es el que más caro sale y el que menos se ve venir. Este juego se define por una frase —*la diplomacia es aritmética*— y una capa de gestión mal dimensionada la sustituye por otra —*el juego va de optimizar tu economía*— sin que nadie tome la decisión en ningún momento. El techo de 5 edificios y 6 Políticas no es una limitación de alcance: es la defensa de la tesis.

18. Vocabulario nuevo

Se añade a [GAME_DESIGN §18](#). Comprobado contra las colisiones que ya tiene el proyecto.

Español	English	Código	Clave i18n
Zona	Zone	zone	ZONE
Solar	Holding	holding	ZONE_HOLDING
Marca	March	march	ZONE_MARCH
Corona	Crown	crown	ZONE_CROWN
Cerco	Ward	ward	ZONE_WARD
Puerta	Gate	gate	ZONE_GATE
Coloso	Colossus	colossus	NPC_COLOSSUS
Despojo	Spoils	spoils	LOOT_SPOILS
Botín	Plunder	plunder	LOOT_PLUNDER
Mena	Vein	vein	MAP_VEIN
Mineral	Ore	ore	RES_ORE
Brasa	Ember	ember	RES_EMBER
Extratora	Extractor	extractor	BLD_EXTRACTOR
Fundición	Foundry	foundry	BLD_FOUNDRY
Arsenal	Arsenal	arsenal	BLD_ARSENAL
Acopio	Depot	depot	BLD_DEPOT
Atalaya	Watch	watch	BLD_WATCH
Grado	Tier	tier	TROOP_TIER
Política	Policy	policy	POLICY
Acto	Act	act	CAMPAIGN_ACT

Colisiones comprobadas

Parecen lo mismo	No lo son
Yacimiento (seam) vs Mena (vein)	El Yacimiento da Ceniza por controlarlo; la Mena da material si construyes encima
Cerco (ward) vs Mortaja (shroud)	El Cerco es la frontera de zona; la Mortaja es la doctrina de ocultación
Corona (zona 3) vs Núcleo (core)	La Corona es la zona; el Núcleo es la región que hay dentro
Brasa (ember) vs Fuego (fire) vs Ceniza (ash)	Material · arma · moneda de victoria. Tres cosas, tres palabras
Despojo (spoils) vs Botín (plunder)	El Despojo cae de un Coloso; el Botín se lo quitas a un jugador
Grado (tier) vs Nivel (level)	El Grado es de tropa; el Nivel es de edificio. Nunca al revés

19. Lo que cambió al construirlo

Esta sección es el registro honesto de la distancia entre la especificación y el juego. Cinco cosas de las secciones anteriores estaban mal, y no se han borrado: se han corregido aquí, con lo que las cazó.

19.1 Cinco correcciones

#	El documento decía	La realidad	Lo cazó
1	El Coloso ocupa <code>gate.inner</code> (§3.3)	Ahí vive al otro lado del Cerco que guarda : nadie llega, nadie lo mata, la Puerta no se abre nunca y la partida es imposible de ganar . Ahora está en <code>gate.outer</code>	Una campaña de 24 turnos en la que ningún bot pisó una Puerta
2	Las Puertas se alinean con el Bastión	Un problema de bien público no se le plantea a nadie si cada uno tiene el suyo en mitad de su casa. Van en la frontera entre sectores	Revisión al arreglar el punto 1
3	Coordinarse contra un Coloso abarata el asedio (§3.4)	Dos asientos en la misma región se aniquilaban entre ellos antes de tocarlo. Entra la tregua: ante un Coloso vivo no hay guerra (ADR-047)	El test «entre dos, cada uno pierde menos»: 40 de pérdida contra 6
4	La Marca reparte Menas de forma asimétrica por sector (§4.3)	Imposible . Bajo rotación C_n todo sector es idéntico por construcción: no puede haber asimetría entre jugadores, y eso es la garantía. El eje de comercio sale de que a todos les falte lo mismo: el Solar da Mineral, la Brasa vive fuera	Escribir <code>spec.ts</code> y ver que la afirmación no podía ser cierta
5	El desgaste del asedio es simétrico	Salía a 45 % de bajas por turno: no era un problema diplomático, era un muro. Lo que le quitas y lo que te quita son ahora constantes distintas	Cero Puertas abiertas en dos campañas completas

Y una que el documento no vio en absoluto: **la economía tenía una trampa de arranque sin salida**. El material solo sale de Extractoras, las Extractoras cuestan material, y quien gastara su capital inicial en otra cosa se quedaba sin economía para el resto de la partida. Ahora toda ciudad se funda sobre una veta (ADR-047).

19.2 Los números, ya medidos

Lo que en el documento iba con  como estimación, medido sobre el juego real:

Magnitud	Estimado	Medido
Regiones, 5 jugadores	271	271
Vista de jugador, serializada	~62 KB	43,2 KB
...de los cuales, el mapa	—	36,3 KB (84 %)
Vista sin el mapa	~24 KB	6,9 KB
Estado de partida	≤ 180 KB	53 KB
Vistas por campaña	~7,4 MB	5,2 MB → 0,85 MB con ADR-044
Regiones en el DOM a la vez	≤ 96	46 (Solar) · 57 (Marca)

El presupuesto de datos de §14.1 se cumple **solo** con ADR-044 aplicado. Sin él, una campaña ocupa 5,2 MB contra los 3 MB de tope, y el free tier da para ~95 campañas archivadas en vez de ~580. Por eso esa ADR dejó de ser una optimización y pasó a ser parte del refactor.

19.3 Una constante recalibrada, por la razón de siempre

`economy.diminishingK` baja de **0,015 a 0,0088**. La «parte justa» pasó de ser el sector entero (19 regiones con cinco jugadores) a ser el Solar (33), así que con la constante vieja la penalización empezaba mucho más tarde: doblar el territorio daba **1,24x** donde el diseño pide 1,55x.

Lo cazó el mismo test que ya lo cazó en la v0.2 — el que fija la **intención declarada** en vez del número. Un test que comprobara `diminishingK === 0.015` habría bendecido el error las dos veces.

19.4 Y un hallazgo que no era del refactor

El checksum del estado volvía a serializar el mapa entero en cada turno: **4,5 ms de los 5,6 que costaba resolver un turno**, sin aportar un bit de información nueva después del turno 0, porque el mapa es inmutable durante toda la partida. Ahora entra por su propio checksum, calculado una vez por mapa.

La garantía no se toca —un mapa distinto sigue dando un estado distinto, y hay dos tests que lo fijan— y las campañas simuladas van dos veces y media más rápidas. Es la misma idea que [ADR-044](#), aplicada al checksum en vez de a la base de datos, y habría merecido la pena aunque este refactor no se hubiera hecho nunca.

19.5 Lo que sigue sin existir

Para que nadie lea este documento y dé por hecho lo que no está:

- Diplomacia: Sellos, Rupturas, Coaliciones y ofertas
- Núcleo y Consagración: la Corona se puede alcanzar, pero no se consagra nada
- El simulador de balance (packages/sim) – sin él, las ~50 constantes son provisionales
- Anomalías, Sombra y doctrinas activas
- Que la Corona se alcance de verdad en 24 turnos: con los rivales actuales no pasa

Ese último punto es una cuestión de calibración, no de motor, y la decide el simulador. Fingirlo en un test habría sido exactamente el tipo de humo que este documento existe para evitar.